University of Exeter

ECM2433: The C Family

Structures and pointers

Overview

- Structures
  - the closest thing C has to a class
  - typedef
  - more pointers!
- Linked lists
  - and other dynamic data structures

Construction: Stephen Doyle
Photo: Grant Cornett

Structures

Structures are

- a type of variable that groups multiple related data items together
- the closest thing C has to a class

Declared like:

```
struct <structure name>
{
   <variable definition>;
   <variable definition>;
};
```

usually before the function prototypes.

Structures: example

```
struct bookStruct
{
   char   title[181] = "";
   char   author[51] = "";
   float  price;
   int    numPages;
};

int main ()
{
   struct bookStruct   myBook;
   struct bookStruct   anotherBook;

   myBook.price = 12.99;
   strlcpy(myBook.title, "The Joy of Programming in C", sizeof(title);
   anotherBook = myBook;
   return 0;
}
```

typedef

Define your own datatypes using the `typedef` command:

```
typedef struct bookStruct BOOK;
```

the actual datatype

the name of the new 'type'

Can be used as a datatype in a variable declaration:

```
BOOK    myBook;
```

which is exactly the same as

```
struct bookStruct    myBook;
```

Other structs and typedefs

Via <stdlib.h>:

```
typedef unsigned long int size_t;      /* As used by strlen, malloc, etc */
```

From object.h in the Python C API

```
typedef struct{
    int slot;    /* slot id, see below */
    void *pfunc; /* function pointer */
} PyType_Slot;

typedef struct{
    const char* name;
    int basicsize;
    int itemsize;
    unsigned int flags;
    PyType_Slot *slots; /* terminated by slot==0. */
} PyType_Spec;
```

Other structs and typedefs: stdio.h

```
typedef struct __sFILE {
        unsigned char *_p;      /* current position in (some) buffer */
        int     _r;             /* read space left for getc() */
        int     _w;             /* write space left for putc() */
        short   _flags;         /* flags, below; this FILE is free if 0 */
        short   _file;          /* fileno, if Unix descriptor, else -1 */
        struct  __sbuf _bf;     /* the buffer (at least 1 byte, if !NULL) */
        int     _lbfsize;       /* 0 or -_bf._size, for inline putc */

        /* operations */
        void    *_cookie;       /* cookie passed to io functions */
        int     (* _Nullable _close)(void *);
        int     (* _Nullable _read) (void *, char *, int);
        fpos_t  (* _Nullable _seek) (void *, fpos_t, int);
        int     (* _Nullable _write)(void *, const char *, int);

        /* separate buffer for long sequences of ungetc() */
        struct  __sbuf _ub;     /* ungetc buffer */
        struct __sFILEX *_extra; /* additions to FILE to not break ABI */
        int     _ur;            /* saved _r when _r is counting ungetc data */

        /* tricks to meet minimum requirements even when malloc() fails */
        unsigned char _ubuf[3]; /* guarantee an ungetc() buffer */
        unsigned char _nbuf[1]; /* guarantee a getc() buffer */

        /* separate buffer for fgetln() when line crosses buffer boundary */
        struct  __sbuf _lb;     /* buffer for fgetln() */

        /* Unix stdio files get aligned to block boundaries on fseek() */
        int     _blksize;       /* stat.st_blksize (may be != _bf._size) */
        fpos_t  _offset;        /* current lseek offset (see WARNING) */
} FILE;
```

structs

Like any other datatype, a function can accept a structure as a parameter:

```
int getPageCount(struct bookStruct newBook)
{ return newBook.numPages; }
```

or return one:

```
struct bookStruct makeBook(...)
{...}
```

structs

Like any other datatype, a function can accept a structure as a parameter:

```
int getPageCount(struct bookStruct newBook)
{ return newBook.numPages; }
```

or return one:

```
struct bookStruct makeBook(...)
{...}
```

Using the new datatype:

```
BOOK makeBook (BOOK myBook)
{...}
```

and you can have arrays of them:

```
BOOK   books[20];
```

Structures and pointers

```
BOOK firstBook;   /* not a pointer */

strlcpy(firstBook.title, "Hidden Figures”, sizeof(Book.title));
strlcpy(firstBook.author, "Margot Lee Shetterly”, sizeof(Book.author));
firstBook.price = 8.99;
firstBook.numPages = 384;
```

Structures and pointers

```
BOOK firstBook;    /* not a pointer */

strlcpy(firstBook.title, "Hidden Figures”, sizeof(Book.title));
strlcpy(firstBook.author, "Margot Lee Shetterly”, sizeof(Book.author));
firstBook.price = 8.99;
firstBook.numPages = 384;
```

```
BOOK *secondBook;      /* a pointer */

if (( secondBook = (BOOK *)malloc(sizeof(BOOK))) == NULL )
{ /* error: not enough memory */ }
strlcpy(secondBook->title, "Hidden Figures”, sizeof(Book.title));
strlcpy(secondBook->author, "Margot Lee Shetterly”, sizeof(Book.author))
secondBook->price = 8.99;
secondBook->numPages = 384;
```

secondBook->price
is shorthand for
(*secondBook).price

Dynamic Data Structures: Arrays

University of Exeter

Arrays are always stored in contiguous memory.

Advantage of arrays:

- can go directly to the *n*th element
- can iterate over them with pointers

Disadvantages of arrays:

- Insertion and deletion requires other elements to be moved
- Size of the array is often unknown in advance. When trying to insert a new el into a full array:
  - either it can't be done
  - or the array must be extended, which will often require the whole array to be moved

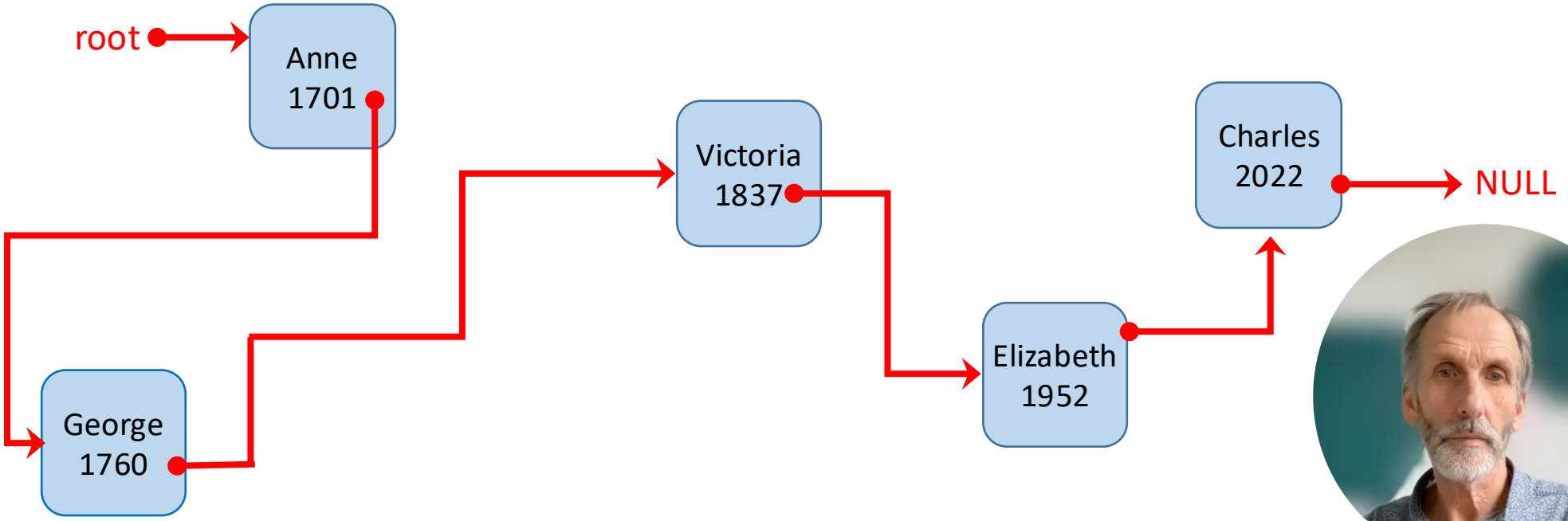
Linked lists

- An ordered sequence of nodes
  each node consists of
  (1) data, and
  (2) one or more links to other nodes to record the sequence
- A memory-efficient means of storing large data structures
  or where the final size of the large data structure is unknown at compile time
- A speed-efficient means of inserting and deleting nodes
- Examples: stacks, queues, trees, circular buffers, ….

Example linked list

```
struct anode
    {   char monarchName[51];
        short int startYear;
        struct anode *next = NULL;
    };
```

Name	Year of accession
George III	1760
Elizabeth II	1952
Anne	1701
Victoria	1837
Charles III	2022

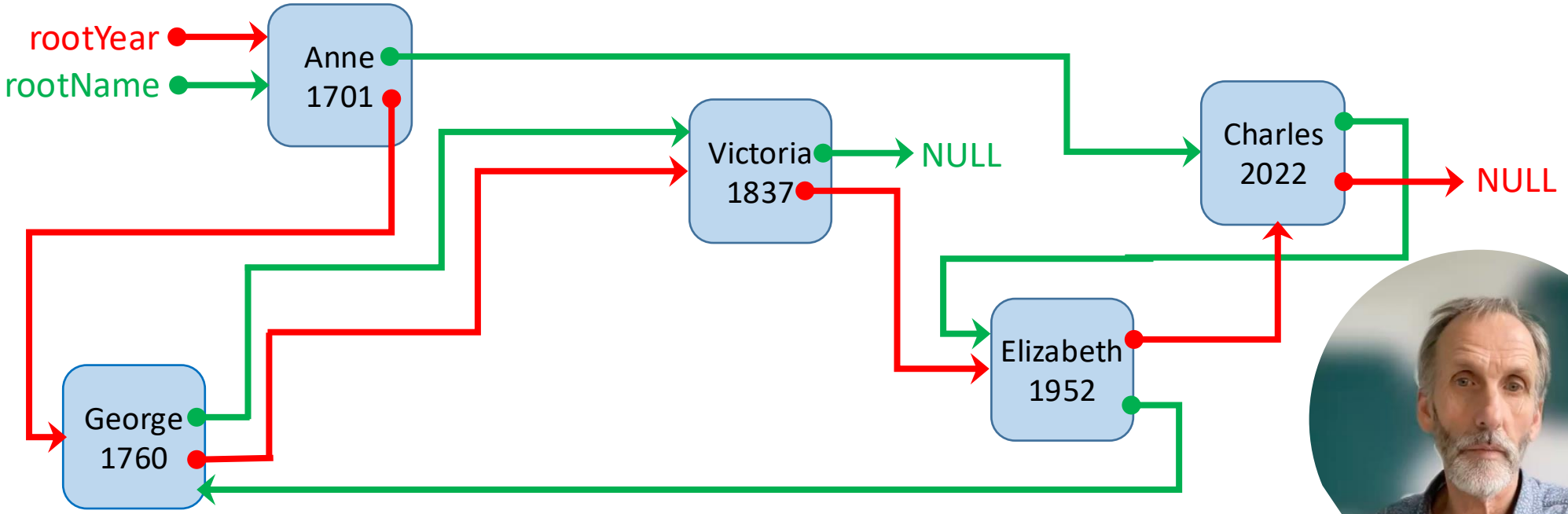


Multiply-linked lists

A doubly-linked list:

```
struct anode
{   char monarchName[51];
    short int startYear;
    struct anode *nextName = NULL;
    struct anode *nextYear = NULL;
};
```

Name	Year of accession
George III	1760
Elizabeth II	1952
Anne	1701
Victoria	1837
Charles III	2022



Dynamic linked lists

University of Exeter

Define the structure of one node in the list:

```
struct anode
{
   /* some data variables here, e.g.: */
   char   data;
   struct anode *nextNode;
};
typedef struct anode NODE;
```

Note: pointer to anode, not anode!

Initialise an empty list:

```
NODE   *root = NULL;
```

Dynamic linked lists

Add the first node to the list

First allocate enough memory:

```
NODE    *newNode;
if ( !(newNode = (NODE *)malloc(sizeof(NODE))))  { /* error */ }
```

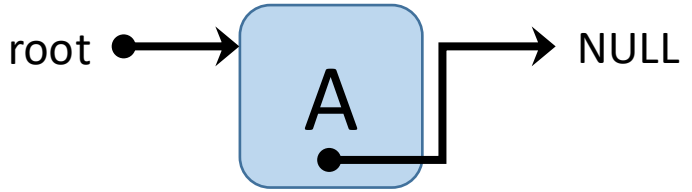
Then set the new node's values:

```
newNode->data     = 'A';
ndeNode->nextNode = NULL;
```

there is no next node at the moment

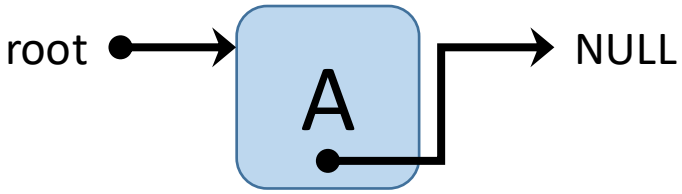
Then add it to the empty list:

```
root = newNode;
```



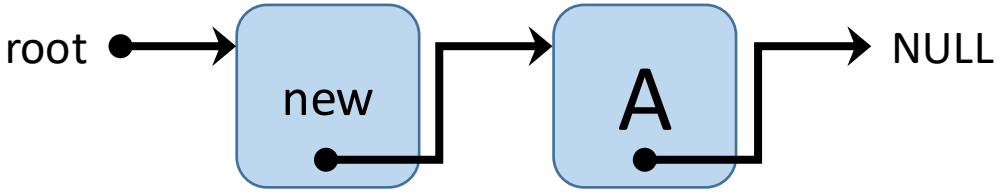
Dynamic linked lists

Add a second new node to this list:



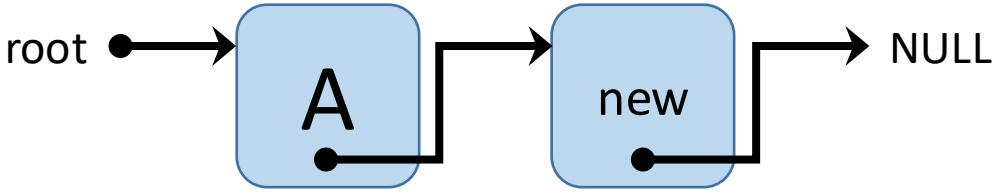
At the beginning of the list:

```
newNode->nextNode = root;
root = newNode;
```



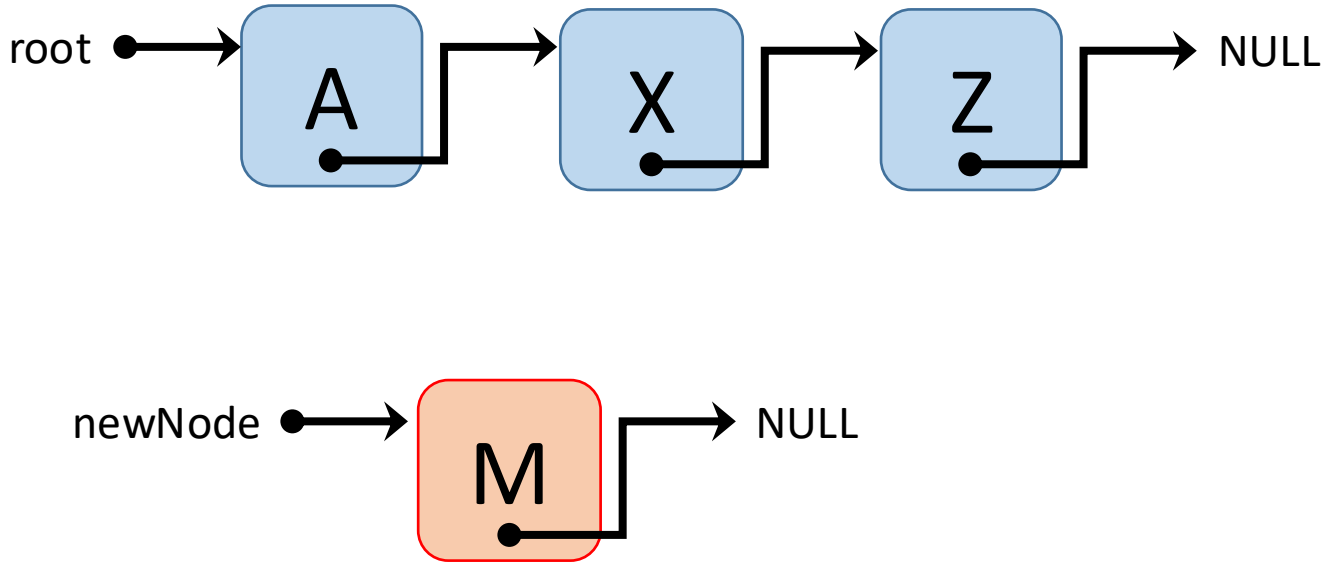
At the end of the list:

```
root->nextNode = newNode;
```



Linked list: insert

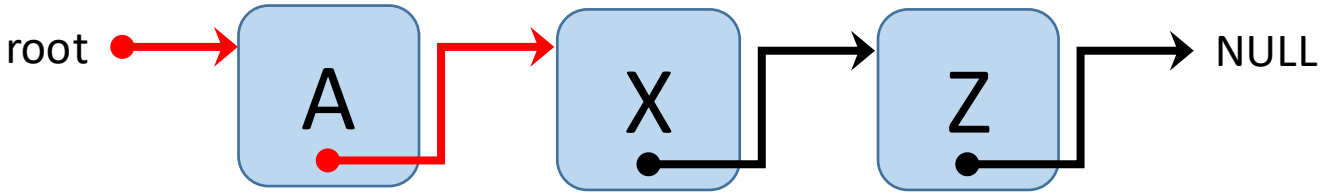
The right way

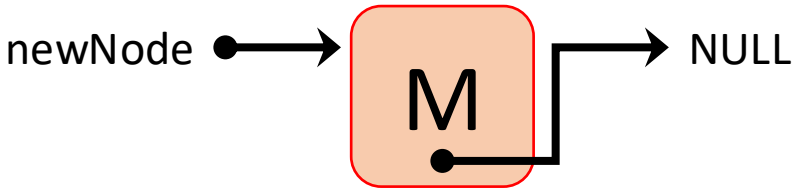


Linked list: insert

The right way

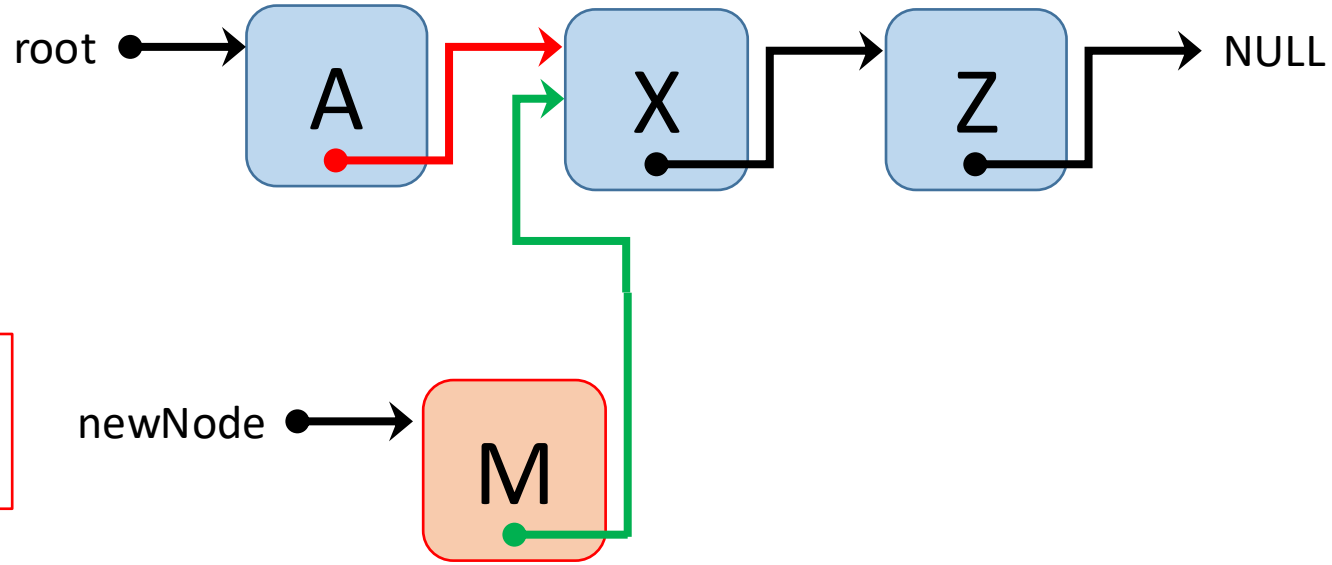
Find the right place to insert





Linked list: insert

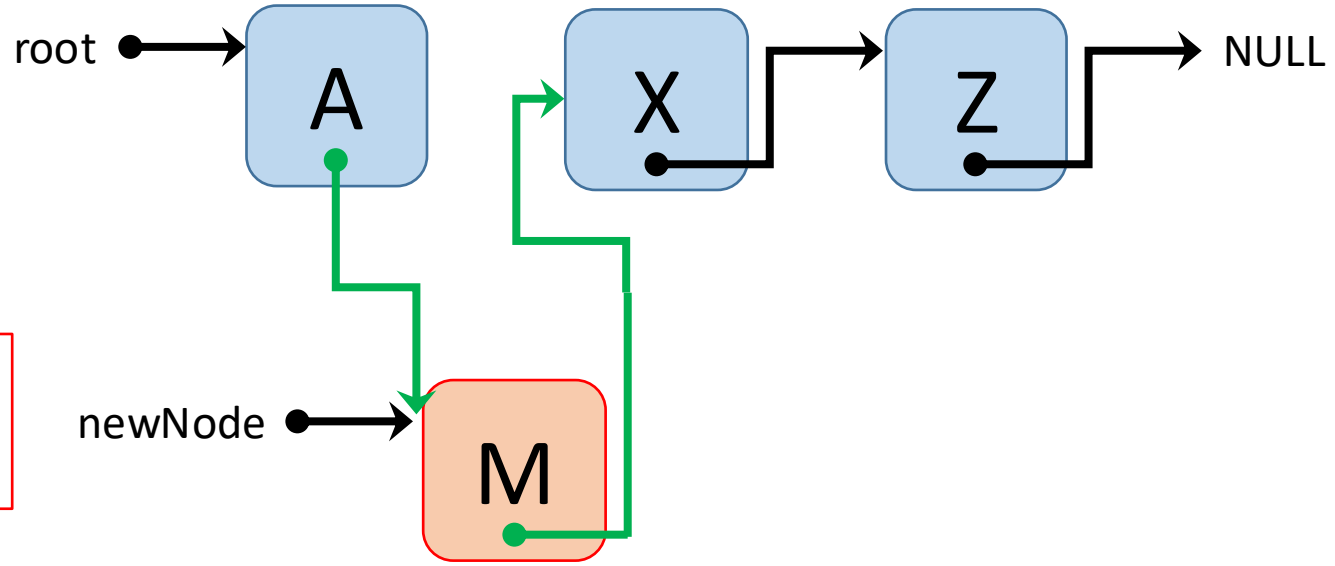
The right way



Link the new node to the tail of the list

Linked list: insert

The right way

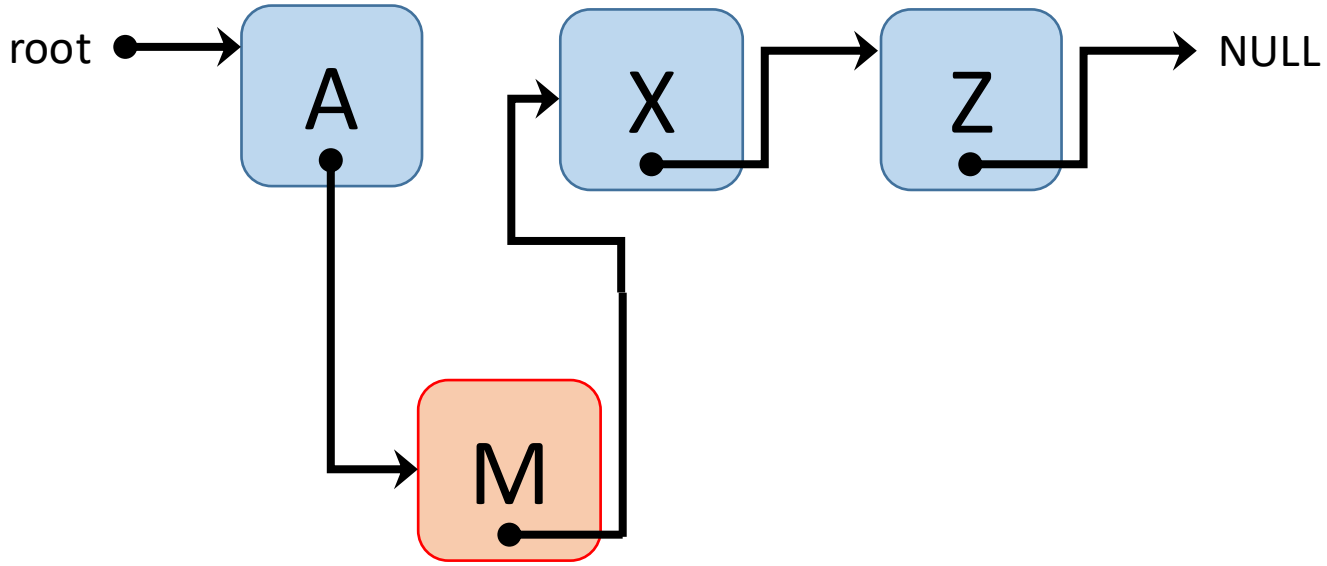


Link the head of the list to the new node

Linked list: insert

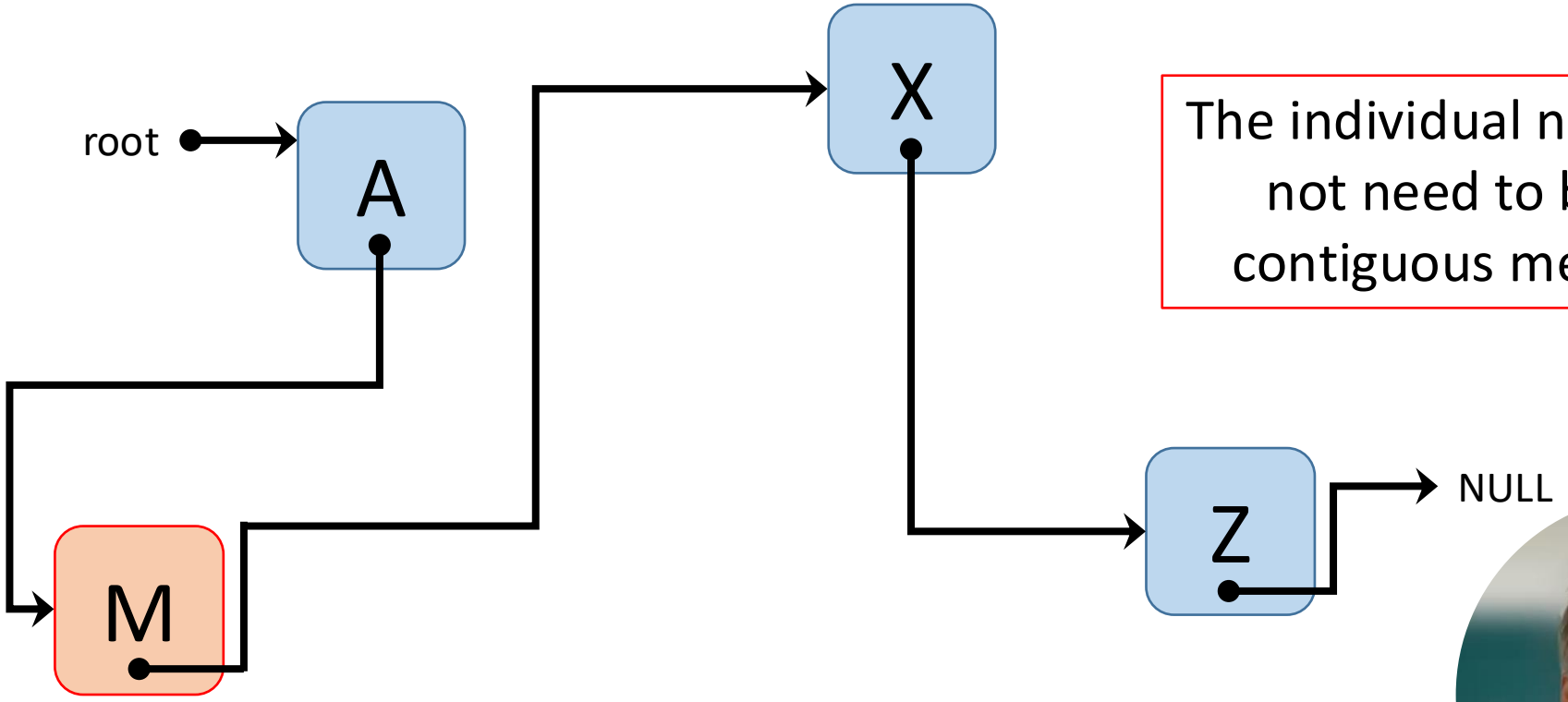
University of Exeter

The right way



Linked list: insert

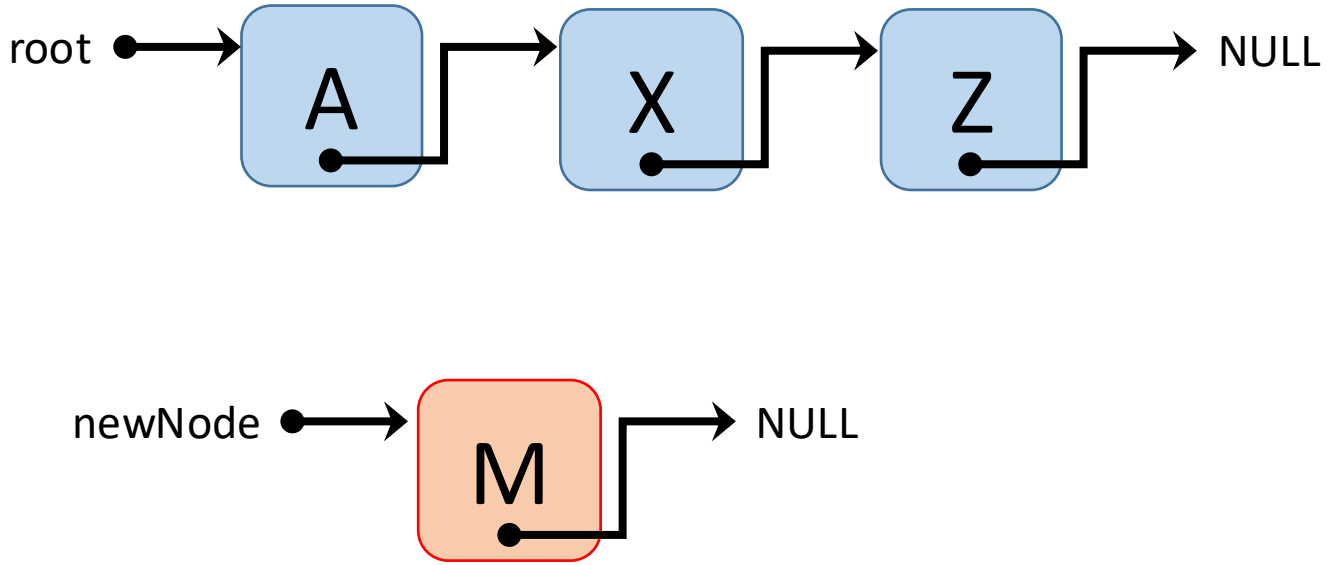
The right way



The individual nodes do not need to be in contiguous memory

Linked list: insert

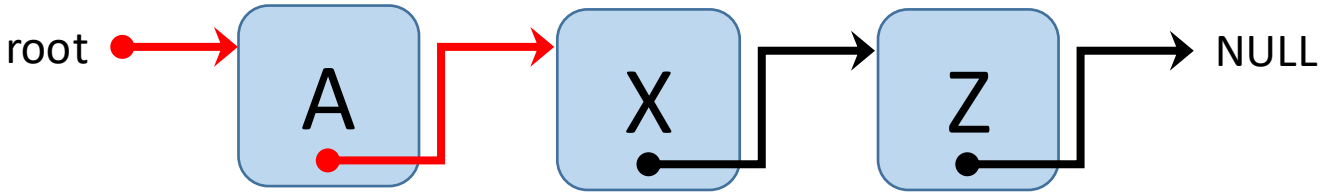
The wrong way

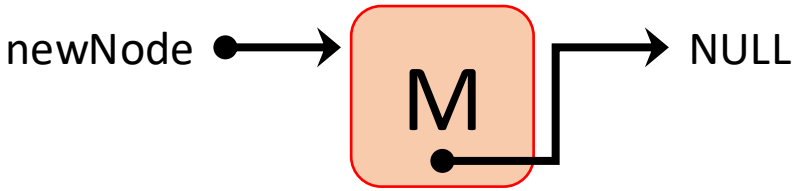


Linked list: insert

The wrong way

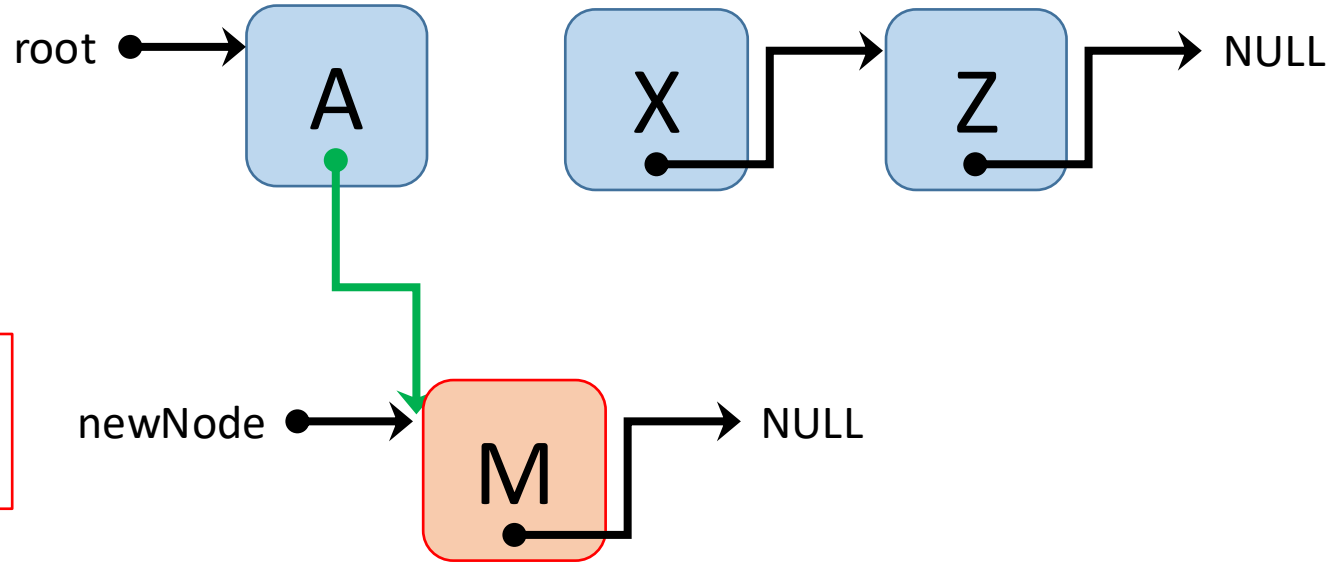
Find the right place to insert





Linked list: insert

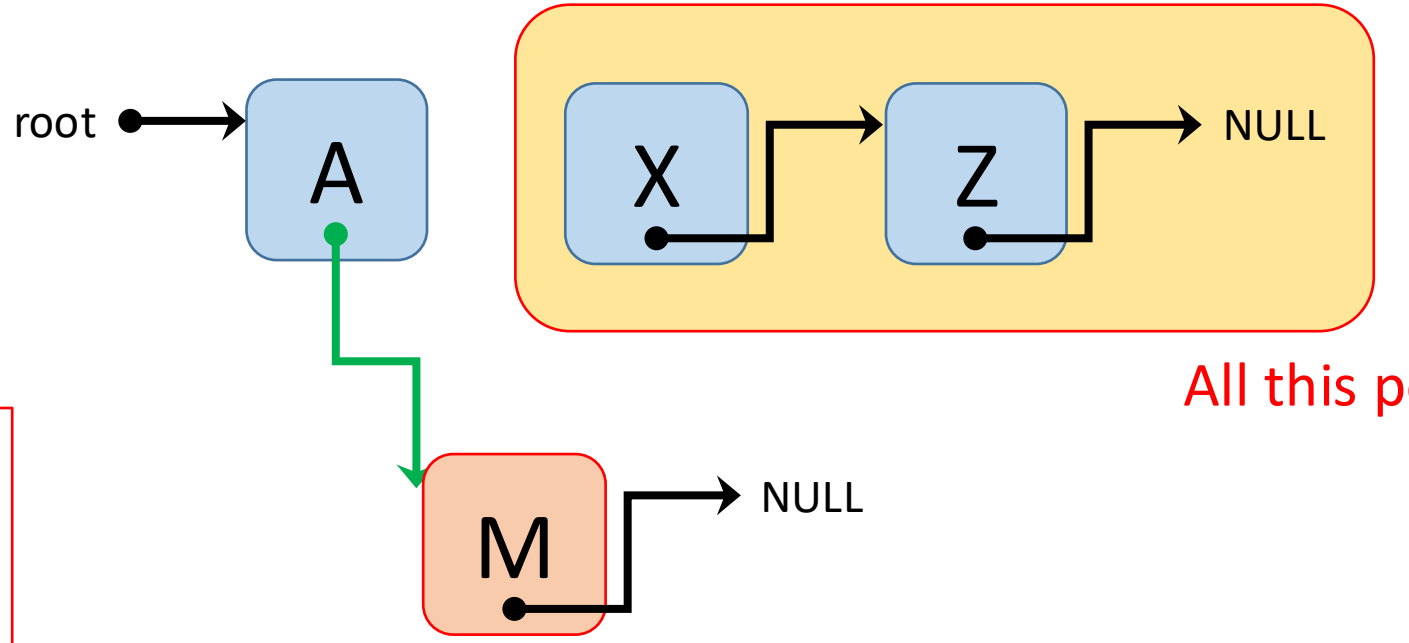
The wrong way



Link the head of the list to the new node

Linked list: insert

The wrong way



All this portion is lost

But now we do not know how to link the new node to the tail of the list

Dynamic linked lists

Advantages of linked lists:

- it does not matter where in memory each node is stored
- insert/delete requires only changing the values of 1 or 2 pointers
- size of linked list is limited only by computer memory
- one list may be linked in more than one way

Disadvantage of linked lists:

- to find an element, must start from the beginning and follow the pointers
  - adding extra pointers can to speed this up

Summary

- Structures
  - the closest thing C has to a class
  - typedef
  - more pointers!
- Linked lists
  - and other dynamic data structures

Construction: Stephen Doyle
Photo: Grant Cornett